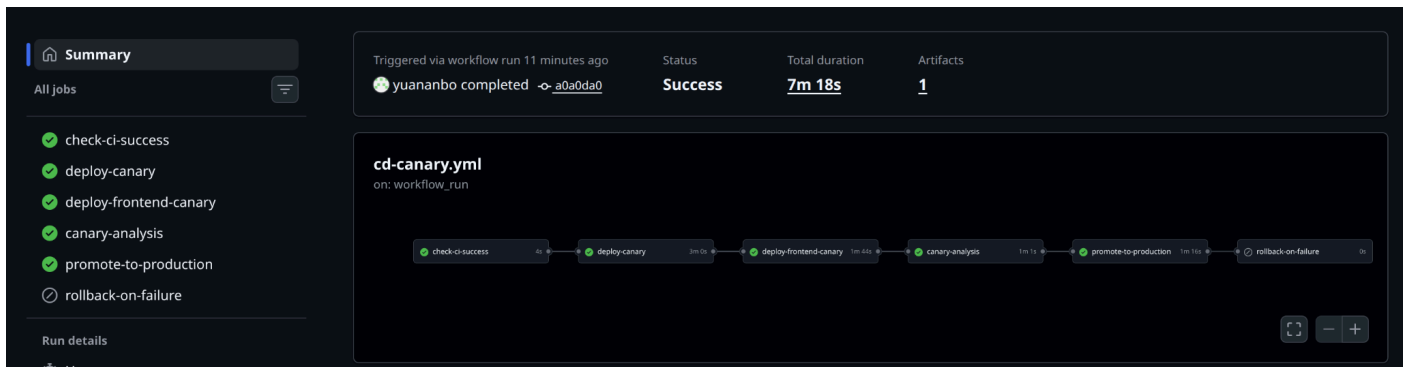


CD pipeline diagram



Order: Check-ci-success -> deploy-canary -> deploy-frontend-canary -> canary-analysis -> promote-to-production -> optional(rollback-on-failure)

Tech Overview:

- We used MongoDB Atlas as our cloud database service to securely store and manage our application's data.
- We leveraged Microsoft Azure as our primary cloud platform to deploy and host our backend environment.
- We used Nginx as our web server to efficiently host and serve the frontend of our application.

Canary Analysis:

After deploying the canary version to our Azure Web Apps, we run a staged canary analysis pipeline before any promotion occurs. First, we verify startup success by checking that both backend (Mobirehab) and frontend (c01mobirehab) are in the Running state. Next, we execute smoke/API checks: the backend must return HTTP 200 on the public /health endpoint, and the frontend root URL must return HTTP 200. We also run a lightweight performance check on the backend health endpoint to capture response latency and flag abnormal slowness. Finally, we perform a short-interval log quality scan by downloading recent backend/frontend logs and checking the latest log output for high-confidence failure signatures (e.g., DetailsLevel: ERR, unhandled exceptions, traceback, panic, OOM). If any of these checks fail, the canary is rejected and the pipeline stops before promotion. After all the steps, we can see if our apps are running, whether the backend and database is reachable, and if there is any error log in our log stream.

Canary Promotion:

Our promotion strategy is a gated full replacement after canary validation. In practice, we first deploy and validate the canary build, and only when all canary-analysis checks pass do we promote by updating the production Web Apps to the same SHA-tagged images(usually the most recent commit) built by CI. This ensures the promoted version is exactly the version that passed analysis. Because we use Web App container updates followed by restart, promotion introduces a short service interruption rather than true traffic-splitting. Based on current startup and probe timings, expected downtime is typically in the range of about 30-120 seconds per app (environment-dependent). During this window, users may briefly experience transient errors or delayed responses, but risk is reduced because only a previously validated canary image is promoted.

Link: <https://github.com/yuananbo/HealthEduLtd/blob/main/.github/workflows/cd-canary.yml>

